

Introduction

1.1 SOFTWARE MODELING

Modeling is used in many walks of life, going back to early civilizations such as Ancient Egypt, Rome, and Greece, where modeling was used to provide small-scale plans in art and architecture (Figure 1.1). Modeling is widely used in science and engineering to provide abstractions of a system at some level of precision and detail. The model is then analyzed in order to obtain a better understanding of the system being developed. According to the Object Modeling Group (OMG), “modeling is the designing of software applications before coding.”

In model-based software design and development, software modeling is used as an essential part of the software development process. Models are built and analyzed prior to the implementation of the system, and are used to direct the subsequent implementation.

A better understanding of a system can be obtained by considering it from different perspectives (also referred to as multiple views) (Gomaa 2006; Gomaa and Shin 2004), such as requirements models, static models, and dynamic models of the software system. A graphical modeling language such as UML helps in developing, understanding, and communicating the different views.

This chapter introduces object-oriented methods and notations, an overview of software modeling and architectural design, and an introduction to model-driven architecture and UML. The chapter then briefly describes the evolution of software design methods, object-oriented analysis and design methods, and concurrent, distributed, and real-time design methods.

1.2 OBJECT-ORIENTED METHODS AND THE UNIFIED MODELING LANGUAGE

Object-oriented concepts are crucial in software analysis and design because they address fundamental issues of software modifiability, adaptation, and evolution. Object-oriented methods are based on the concepts of information hiding, classes, and inheritance. Information hiding can lead to systems that are more self-contained

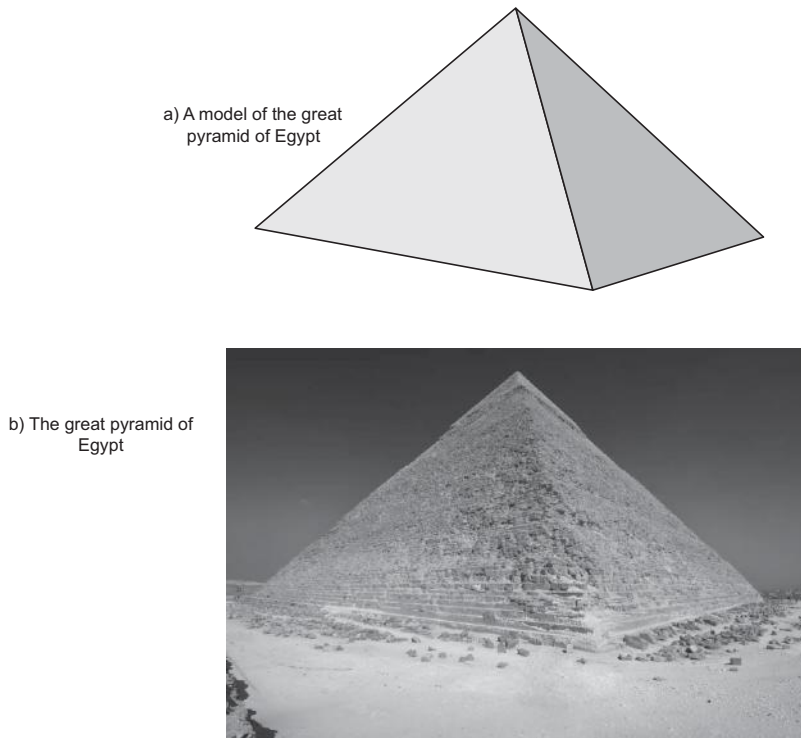


Figure 1.1. Example of modeling and architecture

and hence are more modifiable and maintainable. Inheritance provides an approach for adapting a class in a systematic way.

With the proliferation of notations and methods for the object-oriented analysis and design of software applications, the Unified Modeling Language (UML) was developed to provide a standardized graphical language and notation for describing object-oriented models. However, because UML is methodology-independent, it needs to be used together with an object-oriented analysis and design method. Because the UML is now the standardized graphical language and notation for describing object-oriented models, this book uses the UML notation throughout.

Modern object-oriented analysis and design methods are model-based and use a combination of use case modeling, static modeling, state machine modeling, and object interaction modeling. Almost all modern object-oriented methods use the UML notation for describing software requirements, analysis, and design models (Booch, Rumbaugh, and Jacobson 2005; Fowler 2004; Rumbaugh, Booch, and Jacobson 2005).

In **use case modeling**, the functional requirements of the system are defined in terms of use cases and actors. **Static modeling** provides a structural view of the system. Classes are defined in terms of their attributes, as well as their relationships with other classes. **Dynamic modeling** provides a behavioral view of the system. The use cases are realized to show the interaction among participating objects. Object interaction diagrams are developed to show how objects communicate with each other to realize the use case. The state-dependent aspects of the system are defined with statecharts.

1.3 SOFTWARE ARCHITECTURAL DESIGN

A **software architecture** (Bass, Clements, and Kazman 2003; Shaw and Garlan 1996) separates the overall structure of the system, in terms of components and their interconnections, from the internal details of the individual components. The emphasis on components and their interconnections is sometimes referred to as *programming-in-the-large*, and the detailed design of individual components is referred to as *programming-in-the-small*.

A software architecture can be described at different levels of detail. At a high level, it can describe the decomposition of the software system into subsystems. At a lower level, it can describe the decomposition of subsystems into modules or components. In each case, the emphasis is on the external view of the subsystem/component – that is, the interfaces it provides and requires – and its interconnections with other subsystems/components.

The software quality attributes of a system should be considered when developing the software architecture. These attributes relate to how the architecture addresses important nonfunctional requirements, such as performance, security, and maintainability.

The software architecture is sometimes referred to as a high-level design. A software architecture can be described from different views, as described in Section 1.7. It is important to ensure that the architecture fulfills the software requirements, both functional (what the software has to do) and nonfunctional (how well it should do it). It is also the starting point for the detailed design and implementation, when typically the development team becomes much larger.

1.4 METHOD AND NOTATION

This section defines important terms for software design.

A **software design notation** is a means of describing a software design either graphically or textually, or both. For example, class diagrams are a graphical design notation, and pseudocode is a textual design notation. UML is a graphical notation for object-oriented software applications. A design notation suggests a particular approach for performing a design; however, it does not provide a systematic approach for producing a design.

A **software design concept** is a fundamental idea that can be applied to designing a system. For example, information hiding is a software design concept.

A **software design strategy** is an overall plan and direction for developing a design. For example, object-oriented decomposition is a software design strategy.

Software structuring criteria are heuristics or guidelines used to help a designer in structuring a software system into its components. For example, object structuring criteria provide guidelines for decomposing the system into objects.

A **software design method** is a systematic approach that describes the sequence of steps to follow in order to create a design, given the software requirements of the application. It helps a designer or design team identify the design decisions to be made, the order in which to make them, and the structuring criteria to use in making them. A design method is based on a set of design concepts, employs one or more design strategies, and documents the resulting design, using a design notation.

During a given design step, the method might provide a set of structuring criteria to help the designer in decomposing the system into its components.

The Collaborative Object Modeling and Design Method, or COMET, uses the UML notation to describe the design. COMET is based on the design concepts of information hiding, classes, inheritance, and concurrent tasks. It uses a design strategy of concurrent object design, which addresses the structuring of a software system into active and passive objects and defines the interfaces between them. It provides structuring criteria to help structure the system into objects during analysis, and additional criteria to determine the subsystems and concurrent tasks during design.

1.5 COMET: A UML-BASED SOFTWARE MODELING AND DESIGN METHOD FOR SOFTWARE APPLICATIONS

This book describes a UML-based software modeling and architectural design method called COMET. COMET is an iterative use case driven and object-oriented software development method that addresses the requirements, analysis, and design modeling phases of the software development life cycle. The functional requirements of the system are defined in terms of actors and use cases. Each use case defines a sequence of interactions between one or more actors and the system. A use case can be viewed at various levels of detail. In a *requirements* model, the functional requirements of the system are defined in terms of actors and use cases. In an *analysis* model, the use case is realized to describe the objects that participate in the use case and their interactions. In the *design* model, the software architecture is developed, addressing issues of distribution, concurrency, and information hiding.

1.6 UML AS A STANDARD

This section briefly reviews the evolution of UML into a standard modeling language and notation for describing object-oriented designs. The evolution of UML is described in detail by Kobryn (1999). UML 0.9 unified the modeling notations of Booch, Jacobson (1992), and Rumbaugh et al. (1991). This version formed the basis of a standardization effort, with the additional involvement of a diverse mix of vendors and system integrators. The standardization effort culminated in submission of the initial UML 1.0 proposal to the OMG in January 1997. After some revisions, the final UML 1.1 proposal was submitted later that year and adopted as an object modeling standard in November 1997.

The OMG maintains UML as a standard. The first adopted version of the standard was UML 1.3. There were minor revisions with UML 1.4 and 1.5. A major revision to the notation was made in 2003 with UML 2.0. The latest books on UML conform to UML 2.0, including the revised editions of Booch, Rumbaugh, and Jacobson (2005), Rumbaugh, Booch, and Jacobson (2005), Fowler (2004), Eriksson et al. (2004), and Douglass (2004). There have been minor revisions since then. The current version of the standard is referred to as UML 2.

1.6.1 Model-Driven Architecture with UML

In the OMG's view, "modeling is the designing of software applications before coding." The OMG promotes model-driven architecture as the approach in which UML models of the software architecture are developed prior to implementation. According to the OMG, UML is methodology-independent; UML is a notation for describing the results of an object-oriented analysis and design developed via the methodology of choice.

A UML model can be either a platform-independent model (PIM) or a platform-specific model (PSM). The PIM is a precise model of the software architecture before a commitment is made to a specific platform. Developing the PIM first is particularly useful because the same PIM can be mapped to different middleware platforms, such as COM, CORBA, .NET, J2EE, Web Services, or another Web platform. The approach in this book is to use the concept of model-driven architecture to develop a component-based software architecture, which is expressed as a UML platform-independent model.

1.7 MULTIPLE VIEWS OF SOFTWARE ARCHITECTURE

A software architecture can be considered from different perspectives, which are referred to as different views. Kruchten (Kruchten 1995) introduced the 4+1 view model of software architecture, in which he advocated a multiple-view modeling approach for software architectures, in which the use case view is the unifying view (the 1 view of the 4+1 views). The views are the logical view, which is a static modeling view; the process view, which is a concurrent process or task view; and the development view, which is a subsystem and component design view. Hofmeister et al. (2000) describe an industrial perspective on applied software architecture consisting of four views: a conceptual view, which describes the main design elements and the relationships between them; a code view, which consists of the source code organized into object code, libraries, and directories; a module view, which consists of subsystems and modules; and an execution view, which is a concurrent and distributed execution perspective.

In this book, we will describe and depict the different modeling views of the software architecture in UML. The views are as follows:

- **Use case view.** This view is a functional requirements view, which is an input to develop the software architecture. Each use case describes the sequence of interactions between one or more actors (external users) and the system.
- **Static view.** The architecture is depicted in terms of classes and relationships, which can be associations, whole/part relationships (compositions or aggregations), or generalization/specialization relationships. Depicted on UML class diagrams.
- **Dynamic interaction view.** This view describes the architecture in terms of objects as well as the message communication between them. This view can also be used to depict the execution sequence of specific scenarios. Depicted on UML communication diagrams.

- **Dynamic state machine view.** The internal control and sequencing of a control component can be depicted using a state machine. Depicted on UML statechart diagrams.
- **Structural component view.** The software architecture is depicted in terms of components, which are interconnected through their ports, which in turn support provided and required interfaces. Depicted on UML structured class diagrams.
- **Dynamic concurrent view.** The software architecture is viewed as concurrent components, executing on distributed nodes, and communicating by messages. Depicted on UML concurrent communication diagrams.
- **Deployment view.** This depicts a specific configuration of the distributed architecture with components assigned to hardware nodes. Depicted on UML deployment diagrams.

1.8 EVOLUTION OF SOFTWARE MODELING AND DESIGN METHODS

In the 1960s, programs were often implemented with little or no systematic requirements analysis and design. Graphical notations – in particular, flowcharts – were often used, either as a documentation tool or as a design tool for planning a detailed design prior to coding. Subroutines were originally created as a means of allowing a block of code to be shared by calling it from different parts of a program. They were soon recognized as a means of constructing modular systems and were adopted as a project management tool. A program could be divided up into modules, where each module could be developed by a separate person and implemented as a subroutine or function.

With the growth of structured programming in the early seventies, the ideas of top-down design and stepwise refinement (Dahl 1972) gained prominence as program design methods, with the goal of providing a systematic approach for structured program design. Dijkstra developed one of the first software design methods with the design of the T.H.E. operating system (Dijkstra 1968), which used a hierarchical architecture. This was the first design method to address the design of a concurrent system, namely, an operating system.

In the mid- to late 1970s, two different software design strategies gained prominence: data flow-oriented design and data structured design. The data flow oriented-design approach as used in Structured Design (see Budgen [2003] for an overview) was one of the first comprehensive and well-documented design methods to emerge. The idea was that a better understanding of the functions of the system could be obtained by considering the flow of data through the system. It provided a systematic approach for developing data flow diagrams for a system and then mapping them to structure charts. Structured Design introduced the coupling and cohesion criteria for evaluating the quality of a design. This approach emphasized functional decomposition into modules and the definition of module interfaces. The first part of Structured Design, based on data flow diagram development, was refined and extended to become a comprehensive analysis method, namely, Structured Analysis (see Budgen [2003] for an overview).

An alternative software design approach was that of data structured design. This view was that a full understanding of the problem structure is best obtained from consideration of the data structures. Thus, the emphasis is on first designing the

data structures and then designing the program structures based on the data structures. The two principal design methods to use this strategy were Jackson Structured Programming (Jackson 1983) and the Warnier/Orr method.

In the database world, the concept of separating logical and physical data was key to the development of database management systems. Various approaches were advocated for the logical design of databases, including the introduction of entity-relationship modeling by Chen.

Parnas (1972) made a great contribution to software design with his advocacy of information hiding. A major problem with early systems, even in many of those designed to be modular, resulted from the widespread use of global data, which made these systems prone to error and difficult to change. Information hiding provided an approach for greatly reducing, if not eliminating, global data.

A major contribution for the design of concurrent and real-time systems came in the late 1970s with the introduction of the MASCOT notation and later the MASCOT design method. Based on a data flow approach, MASCOT formalized the way tasks communicate with each other, either through channels for message communication or through pools (information-hiding modules that encapsulate shared data structures). The data maintained by a channel or pool are accessed by a task only indirectly by calling access procedures provided by the channel or pool. The access procedures also synchronize access to the data, typically using semaphores, so that all synchronization issues are hidden from the calling task.

There was a general maturation of software design methods in the 1980s, and several system design methods were introduced. Parnas's work with the Naval Research Lab (NRL), in which he explored the use of information hiding in large-scale software design, led to the development of the Naval Research Lab Software Cost Reduction Method (Parnas, Clements, and Weiss 1984). Work on applying Structured Analysis and Structured Design to concurrent and real-time systems led to the development of Real-Time Structured Analysis and Design (RTSAD) (see Gomaa [1993] for an overview) and the Design Approach for Real-Time Systems (DARTS) (Gomaa 1984) methods.

Another software development method to emerge in the early 1980s was Jackson System Development (JSD) (Jackson 1983). JSD views a design as being a simulation of the real world and emphasizes modeling entities in the problem domain by using concurrent tasks. JSD was one of the first methods to advocate that the design should model reality first and, in this respect, predated the object-oriented analysis methods. The system is considered a simulation of the real world and is designed as a network of concurrent tasks, in which each real-world entity is modeled by means of a concurrent task. JSD also defied the then-conventional thinking of top-down design by advocating a middle-out behavioral approach to software design. This approach was a precursor of object interaction modeling, an essential aspect of modern object-oriented development.

1.9 EVOLUTION OF OBJECT-ORIENTED ANALYSIS AND DESIGN METHODS

In the mid- to late 1980s, the popularity and success of object-oriented programming led to the emergence of several object-oriented design methods, including

Booch, Wirfs-Brock, Wilkerson, and Wiener (1990), Rumbaugh et al. (1991), Shlaer and Mellor (1988, 1992), and Coad and Yourdon (1991, 1992). The emphasis in these methods was on modeling the problem domain, information hiding, and inheritance.

Parnas advocated using information hiding as a way to design modules that were more self-contained and therefore could be changed with little or no impact on other modules. Booch introduced object-oriented concepts into design initially with information hiding, in the **object-based design** of Ada-based systems and later extended this to using information hiding, classes, and inheritance in **object-oriented design**. Shlaer and Mellor (1988), Coad and Yourdon (1991), and others introduced object-oriented concepts into analysis. It is generally considered that the object-oriented approach provides a smoother transition from analysis to design than the functional approach.

Object-oriented analysis methods apply object-oriented concepts to the analysis phase of the software life cycle. The emphasis is on identifying real-world objects in the problem domain and mapping them to software objects. The initial attempt at object modeling was a static modeling approach that had its origins in information modeling, in particular, entity-relationship (E-R) modeling or, more generally, semantic data modeling, as used in logical database design. Entities in E-R modeling are information-intensive objects in the problem domain. The entities, the attributes of each entity, and relationships among the entities, are determined and depicted on E-R diagrams; the emphasis is entirely on data modeling. During design, the E-R model is mapped to a database, usually relational. In object-oriented analysis, objects in the problem domain are identified and modeled as software classes, and the attributes of each class, as well as the relationships among classes, are determined (Coad 1991; Rumbaugh et al. 1991; Shlaer and Mellor 1988).

The main difference between *classes* in static object-oriented modeling and *entity types* in E-R modeling is that classes have operations but entity types do not have operations. In addition, whereas information modeling only models persistent entities that are to be stored in a database, other problem domain classes are also modeled in static object modeling. The advanced information modeling concepts of aggregation and generalization/specialization are also used. The most widely used notation for static object modeling before UML was the Object Modeling Technique (OMT) (Rumbaugh et al. 1991).

Static object modeling was also referred to as *class modeling* and *object modeling* because it involves determining the classes to which objects belong and depicting classes and their relationships on class diagrams. The term *domain modeling* is also used to refer to static modeling of the problem domain (Rosenberg and Scott 1999; Shlaer and Mellor 1992).

The early object-oriented analysis and design methods emphasized the structural aspects of software development through information hiding and inheritance but neglected the dynamic aspects. A major contribution by the OMT (Rumbaugh et al. 1991) was to clearly demonstrate that dynamic modeling was equally important. In addition to introducing the static modeling notation for the object diagrams, OMT showed how dynamic modeling could be performed with statecharts for showing the state-dependent behavior of active objects and with sequence diagrams to show the sequence of interactions between objects. Rumbaugh et al. (1991) used statecharts,

which are hierarchical state transition diagrams originally conceived by Harel (1988, 1998), for modeling active objects. Shlaer and Mellor (1992) also used state transition diagrams for modeling active objects. Booch initially used object diagrams to show the instance-level interactions among objects and later sequentially numbered the interactions to more clearly depict the communication among objects.

Jacobson (1992) introduced the use case concept for modeling the system's functional requirements. Jacobson also used the sequence diagram to describe the sequence of interactions between the objects that participate in a use case. The use case concept was fundamental to all phases of Jacobson's object-oriented software engineering life cycle. The use case concept has had a profound impact on modern object-oriented software development.

Prior to UML, there were earlier attempts to unify the various object-oriented methods and notations, including Fusion (Coleman et al. 1993) and the work of Texel and Williams (1997). The UML notation was originally developed by Booch, Jacobson, and Rumbaugh to integrate the notations for use case modeling, static modeling, and dynamic modeling (using statecharts and object interaction modeling), as described in Chapter 2. Other methodologists also contributed to the development of UML. An interesting discussion of how UML has evolved and how it is likely to evolve in the future is given in Cobryn [1999] and Selic (1999).

1.10 SURVEY OF CONCURRENT, DISTRIBUTED, AND REAL-TIME DESIGN METHODS

The Concurrent Design Approach for Real-Time Systems (CODARTS) method (Gomaa 1993) built on the strengths of earlier concurrent design, real-time design, and early object-oriented design methods by emphasizing both information-hiding module structuring and concurrent task structuring.

Octopus (Awad, Kuusela, and Ziegler 1996) is a real-time design method based on use cases, static modeling, object interactions, and statecharts. For real-time systems, ROOM (Selic, Gullekson, and Ward 1994) is an object-oriented real-time design method that is closely tied in with a **CASE** (Computer-Assisted Software Engineering) tool called ObjecTime. ROOM is based around actors, which are active objects that are modeled using a variation on statecharts called ROOMcharts. A ROOM model is capable of being executed and thus could be used as an early prototype of the system.

Buhr (1996) introduced an interesting concept called the use case map (based on the use case concept) to address the issue of dynamic modeling of large-scale systems.

For UML-based real-time software development, Douglass (2004, 1999) has provided a comprehensive description of how UML can be applied to real-time systems.

An earlier version of the COMET method for designing concurrent, real-time, and distributed applications, which is based on UML 1.3, is described in Gomaa (2000). This new textbook expands on the COMET method by basing it on UML 2, increasing the emphasis on software architecture, and addressing a wide range of software applications, including object-oriented software architectures, client/server software architectures, service-oriented architectures, component-based software

architectures, concurrent and real-time software architectures, and software product line architectures.

1.11 SUMMARY

This chapter introduced object-oriented methods and notations, software architectural design, and UML. The chapter briefly described the evolution of software design methods, object-oriented analysis and design methods, and concurrent, distributed, and real-time design methods. Chapter 2 provides an overview of the UML notation. Chapter 3 describes software life cycles and methods. Chapter 4 describes software design and architecture concepts. Chapter 5 describes the use case-based software life cycle for the COMET method.

EXERCISES

Multiple-choice questions: For each question, choose one of the answers.

1. What is software modeling?
 - (a) Developing models of software.
 - (b) Designing software applications before coding.
 - (c) Developing software diagrams.
 - (d) Developing software prototypes.
2. What is the Unified Modeling Language?
 - (a) A programming language for describing object-oriented models.
 - (b) A diagramming tool for drawing object-oriented models.
 - (c) A graphical language for describing object-oriented models.
 - (d) A standardized graphical language and notation for describing object-oriented models.
3. What is a software architecture?
 - (a) The software inside a building.
 - (b) The structure of a client/server system.
 - (c) The overall structure of a software system.
 - (d) The software classes and their relationships.
4. What is a software design notation?
 - (a) Notes about the software design.
 - (b) A graphical or textual description of the software.
 - (c) Documentation of the software.
 - (d) A systematic approach for producing a design.
5. What is a software design concept?
 - (a) A graphical or textual description of the software.
 - (b) Documentation of the software.
 - (c) A fundamental idea that can be applied to designing a system.
 - (d) A systematic approach for producing a design.
6. What is a software design strategy?
 - (a) A graphical or textual description of the software.
 - (b) A fundamental idea that can be applied to designing a system.
 - (c) A systematic approach for producing a design.
 - (d) An overall plan and direction for developing a design.
7. What are software structuring criteria?
 - (a) Fundamental ideas that can be applied to designing a system.
 - (b) Systematic approaches for producing a design.
 - (c) Guidelines used to help in structuring a software system into its components.
 - (d) Overall plans for developing a design.
8. What is a software design method?
 - (a) A systematic approach for producing a design.
 - (b) Guidelines used to help in structuring a software system into its components.
 - (c) An overall plan for developing a design.

- (d) A graphical or textual description of the software.
9. What is a platform-independent model (PIM)?
- (a) A software platform before a commitment is made to a specific hardware platform.
 - (b) A precise model of the software architecture before a commitment is made to a specific platform.
 - (c) A precise model of the software architecture mapped to a specific platform.
10. What is a platform-specific model (PSM)?
- (a) A specific hardware platform.
 - (b) A precise model of the software architecture before a commitment is made to a specific platform.
 - (c) A precise model of the software architecture mapped to a specific platform.
 - (d) A graphical or textual description of the software.

Overview of the UML Notation

The notation used for the COMET method is the Unified Modeling Language (UML). This chapter provides a brief overview of the UML notation. The UML notation has evolved since it was first adopted as a standard in 1997. A major revision to the standard was made in 2003, so the current version of the standard is UML 2. The previous versions of the standard are referred to as UML 1.x.

The UML notation has grown substantially over the years, and it supports many diagrams. The approach taken in this book is the same as Fowler's (2004), which is to use only those parts of the UML notation that provide a distinct benefit. This chapter describes the main features of the UML notation that are particularly suited to the COMET method. The purpose of this chapter is not to be a full exposition of UML, because several detailed books exist on this topic, but rather to provide a brief overview. The main features of each of the UML diagrams used in this book are briefly described, but lesser-used features are omitted. The differences between UML 2 notation and UML 1.x notation are also briefly explained.

2.1 UML DIAGRAMS

The UML notation supports the following diagrams for application development:

- **Use case diagram**, briefly described in Section 2.2.
- **Class diagram**, briefly described in Section 2.4.
- **Object diagram** (an instance version of the class diagram), which is not used by COMET.
- **Communication diagram**, which in UML 1.x was called the *collaboration diagram*, briefly described in Section 2.5.1.
- **Sequence diagram**, briefly described in Section 2.5.2.
- **State Machine diagram**, briefly described in Section 2.6.
- **Activity diagram**, which is not used extensively by COMET, is described briefly in [Chapter 6](#).

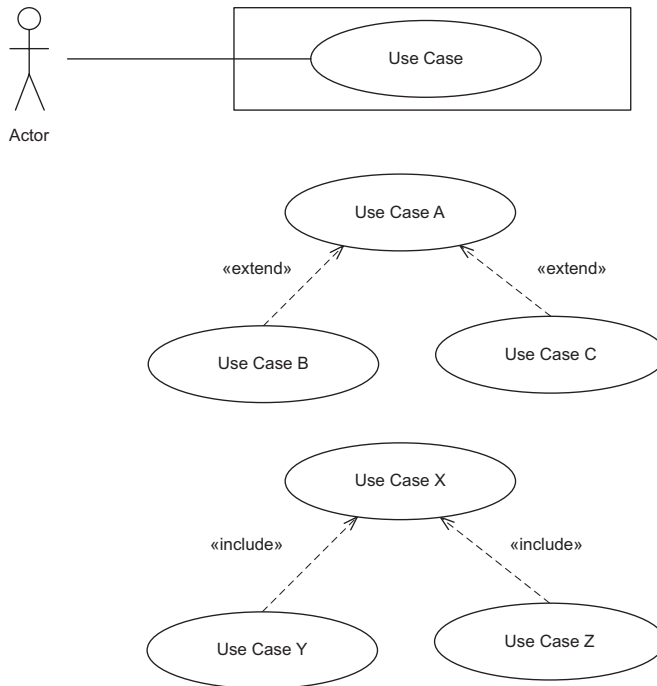


Figure 2.1. UML notation for a use case diagram

- **Composite structure diagram**, a new diagram introduced in UML 2 that is actually better suited for modeling distributed components in a UML platform-independent model. The composite structure diagram is described in [Chapter 17](#).
- **Deployment diagram**, briefly described in Section 2.9.

Chapters 6 through 19 describe how these UML diagrams are used by the COMET method.

2.2 USE CASE DIAGRAMS

An **actor** initiates a use case. A **use case** defines a sequence of interactions between the actor and the system. An actor is depicted as a stick figure on a use case diagram. The system is depicted as a box. A use case is depicted as an ellipse inside the box. Communication associations connect actors with the use cases in which they participate. Relationships among use cases are defined by means of *include* and *extend* relationships. The notation is depicted in [Figure 2.1](#).

2.3 CLASSES AND OBJECTS

Classes and objects are depicted as boxes in the UML notation, as shown in [Figure 2.2](#). The class box always holds the class name. Optionally, the attributes and operations of a class may also be depicted. When all three are depicted, the top

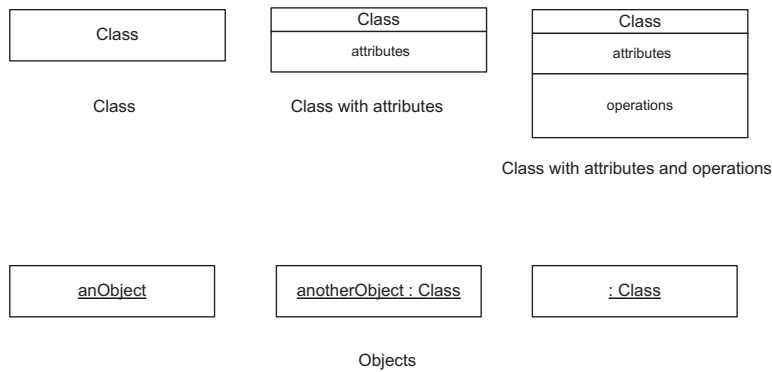


Figure 2.2. UML notation for objects and classes

compartment of the box holds the class name, the middle compartment holds the attributes, and the bottom compartment holds the operations.

To distinguish between a class (the type) and an object (an instance of the type), an object name is shown underlined. An object can be depicted in full with the object name separated by a colon from the class name – for example, anObject : Class. Optionally, the colon and class name may be omitted, leaving just the object name – for example, anObject. Another option is to omit the object name and depict just the class name after the colon, as in : Class. Classes and objects are depicted on various UML diagrams, as described in Section 2.4.

2.4 CLASS DIAGRAMS

In a **class diagram**, classes are depicted as boxes, and the static (i.e., permanent) relationships between them are depicted as lines connecting the boxes. The following three main types of relationships between classes are supported: associations, whole/part relationships, and generalization/specialization relationships, as shown in Figure 2.3. A fourth relationship, the dependency relationship, is often used to show how packages are related, as described in Section 2.7.

2.4.1 Associations

An **association** is a static, structural relationship between two or more classes. An association between two classes, which is referred to as a *binary association*, is depicted as a line joining the two class boxes, such as the line connecting the ClassA box to the ClassB box in Figure 2.3a. An association has a name and optionally a small black arrowhead to depict the direction in which the association name should be read. On each end of the association line joining the classes is the multiplicity of the association, which indicates how many instances of one class are related to an instance of the other class. Optionally, a stick arrow may also be used to depict the direction of navigability.

The **multiplicity** of an association specifies how many instances of one class may relate to a single instance of another class (see Figure 2.3a, right). The multiplicity

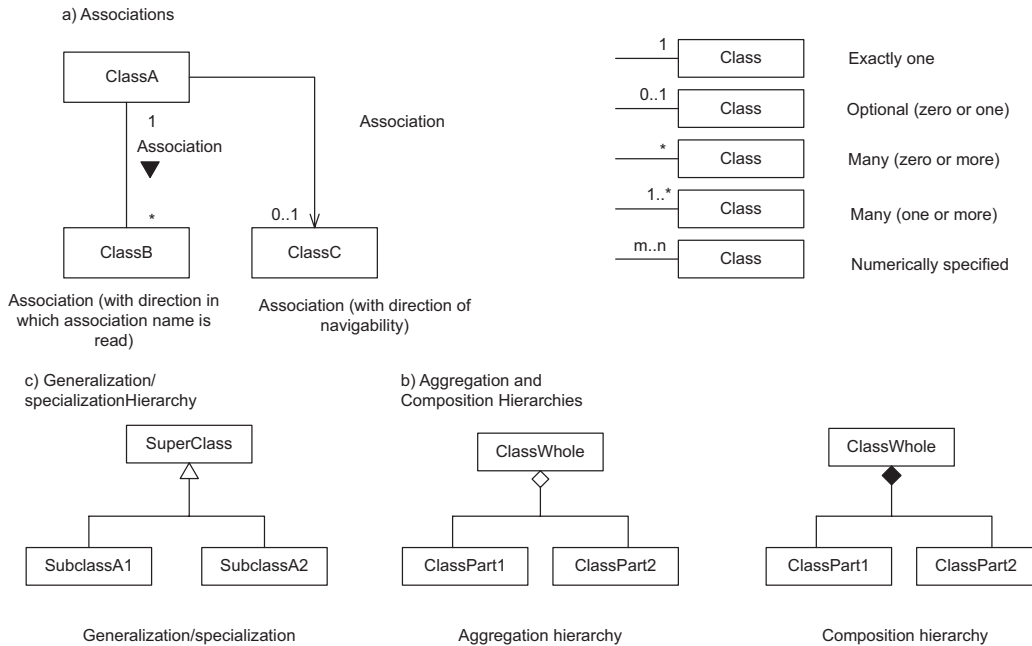


Figure 2.3. UML notation for relationships on a class diagram

of an association can be exactly one (1), optional (0..1), zero or more (*), one or more (1..*), or numerically specified (m..n), where m and n have numeric values.

2.4.2 Aggregation and Composition Hierarchies

Aggregation and composition hierarchies are **whole/part** relationships. The composition relationship (shown by a black diamond) is a stronger form of whole/part relationship than the aggregation relationship (shown by a hollow diamond). The diamond touches the aggregate or composite (Class Whole) class box (see [Figure 2.3b](#)).

2.4.3 Generalization/Specialization Hierarchy

A generalization/specialization hierarchy is an **inheritance** relationship. A generalization is depicted as an arrow joining the subclass (child) to the superclass (parent), with the arrowhead touching the superclass box (see [Figure 2.3c](#)).

2.4.4 Visibility

Visibility refers to whether an element of the class is visible from outside the class, as depicted in [Figure 2.4](#). Depicting visibility is optional on a class diagram. **Public visibility**, denoted with a + symbol, means that the element is visible from outside the class. **Private visibility**, denoted with a – symbol, means that the element is visible only from within the class that defines it and is thus hidden from other classes. **Protected visibility**, denoted with a # symbol, means that the element is visible from within the class that defines it and within all subclasses of the class.

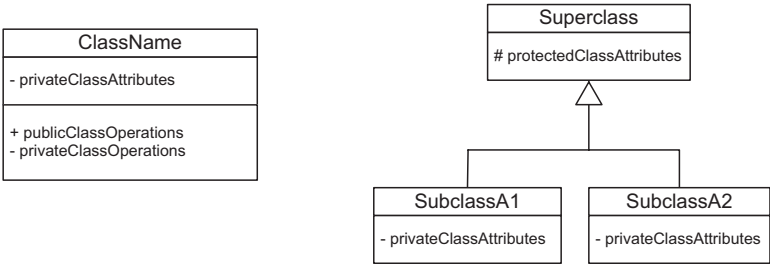


Figure 2.4. UML notation for visibility on a class diagram

2.5 INTERACTION DIAGRAMS

UML has two main kinds of interaction diagrams, which depict how objects interact: the communication diagram and the sequence diagram. On these interaction diagrams, objects are depicted in rectangular boxes. However, object names are not underlined. The main features of these diagrams are described in Sections 2.5.1 and 2.5.2.

2.5.1 Communication Diagrams

A **communication diagram**, which was called a *collaboration diagram* in UML 1.x, shows how cooperating objects dynamically interact with each other by sending and receiving messages. The diagram depicts the structural organization of the objects that interact. Objects are shown as boxes, and lines joining boxes represent object interconnection. Labeled arrows adjacent to the arcs indicate the name and direction of message transmission between objects. The sequence of messages passed between the objects is numbered. The notation for communication diagrams is illustrated in Figure 2.5. An optional iteration is indicated by an asterisk (*), which means that a message is sent more than once. An optional condition means that the message is sent only if the condition is true.

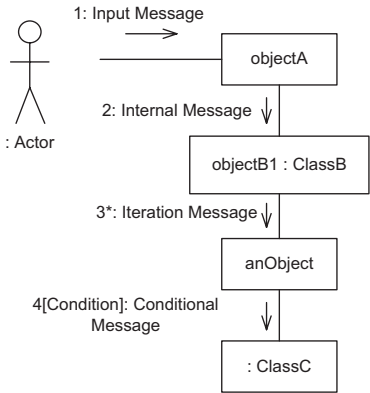


Figure 2.5. UML notation for a communication diagram

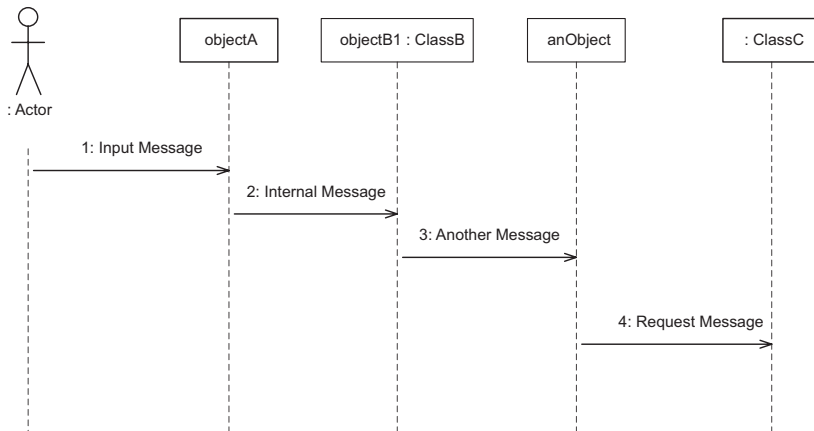


Figure 2.6. UML notation for a sequence diagram

2.5.2 Sequence Diagrams

A different way of illustrating the interaction among objects is to show them on a sequence diagram, which depicts object interaction arranged in time sequence, as shown in Figure 2.6. A **sequence diagram** is a two-dimensional diagram in which the objects participating in the interaction are depicted horizontally and the vertical dimension represents time. Starting at each object box is a vertical dashed line, referred to as a *lifeline*. Optionally, each lifeline has an activation bar (not shown), depicted as a double solid line, which shows when the object is executing.

The actor is usually shown at the extreme left of the page. Labeled horizontal arrows represent messages. Only the source and destination of the arrow are relevant. The message is sent from the source object to the destination object. Time increases from the top of the page to the bottom. The spacing between messages is not relevant.

UML 2 has substantially extended the notation for sequence diagrams to allow for loops and conditionals, as described in Chapters 9 and 11.

2.6 STATE MACHINE DIAGRAMS

In the UML notation, a state transition diagram is referred to as a **state machine diagram**. In this book, the shorter term **statechart** is generally used. In the UML notation, states are represented by rounded boxes, and transitions are represented by arcs that connect the rounded boxes, as shown in Figure 2.7. The initial state of the statechart is depicted by an arc originating from a small black circle. Optionally, a final state may be depicted by a small black circle inside a larger white circle, sometimes referred to as a *bull's-eye*. A statechart may be hierarchically decomposed such that a composite state is broken down into substates.

On the arc representing the state transition, the notation *Event [condition]/Action* is used. The **event** causes the state transition. The optional Boolean **condition** must be true, when the event occurs, for the transition to take place. The

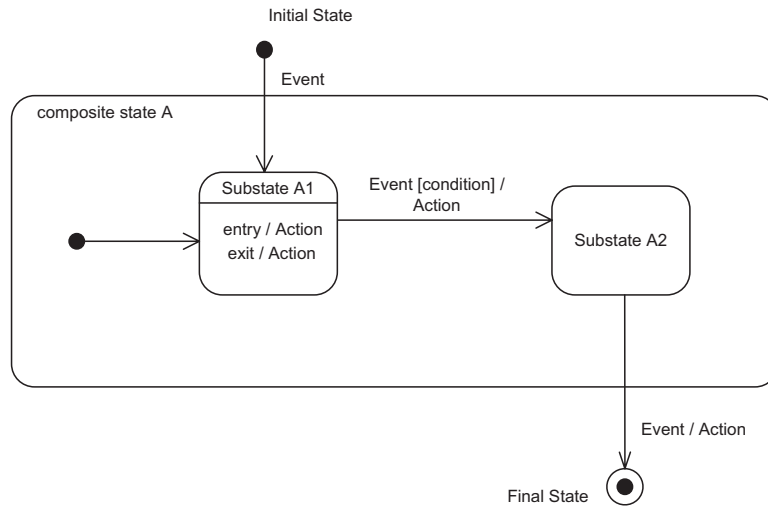


Figure 2.7. UML notation for a state machine: composite state with sequential substates

optional **action** is performed as a result of the transition. Optionally, a state may have any of the following:

- An **entry action**, performed when the state is entered
- An **exit action**, performed on exit from the state

Figure 2.7 depicts a composite state A decomposed into sequential substates A1 and A2. In this case, the statechart is in only one substate at a time; that is, first substate A1 is entered and then substate A2. Figure 2.8 depicts a composite state B decomposed into orthogonal regions BC and BD. In this case, the statechart is in each of the orthogonal regions, BC and BD, at the same time. Each orthogonal

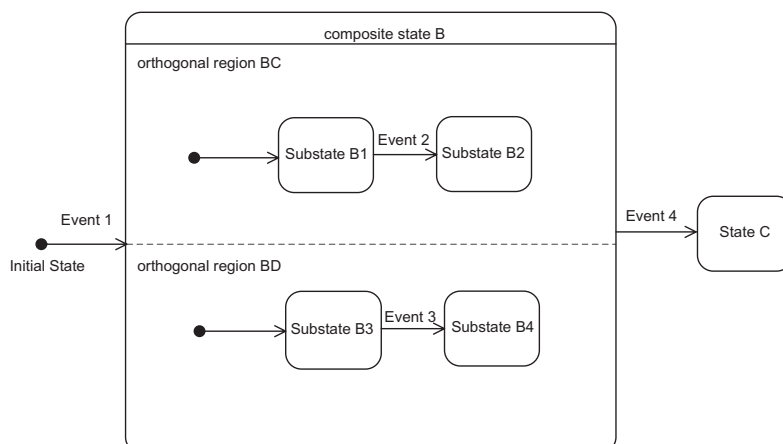


Figure 2.8. UML notation for a state machine: composite state with orthogonal substates

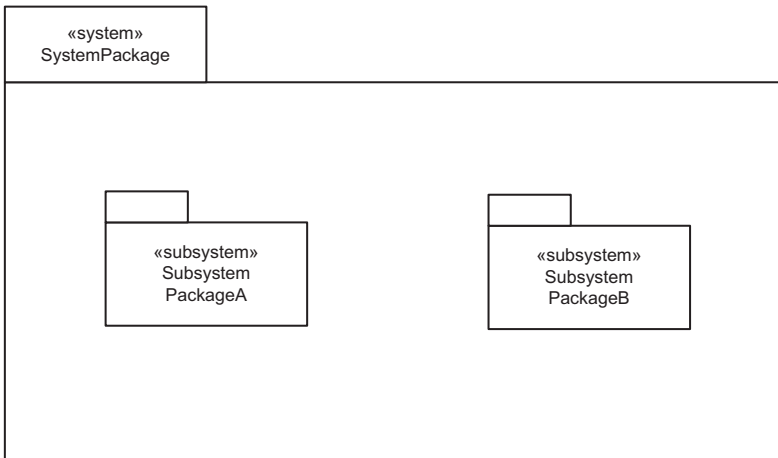


Figure 2.9. UML notation for packages

substate is further decomposed into sequential substates. Thus, when the composite B is initially entered, each of the substates B1 and B3 is also entered.

2.7 PACKAGES

In UML, a **package** is a grouping of model elements – for example, to represent a system or subsystem. A package is depicted by a folder icon, a large rectangle with a small rectangle attached on one corner, as shown in Figure 2.9. Packages may also be nested within other packages. Possible relationships between packages are dependency (shown in Figure 2.9) and generalization/specialization relationships. Packages may be used to contain classes, objects, or use cases.

2.8 CONCURRENT COMMUNICATION DIAGRAMS

In the UML notation, an active object can be used to depict a *concurrent object*, *process*, *thread*, or *task*, and is depicted by a rectangular box with two vertical parallel lines on the left- and right-hand sides. An **active object** has its own thread of control and executes concurrently with other objects. By contrast, a **passive object** has no thread of control. A passive object executes only when another object (active or passive) invokes one of its operations.

Active objects are depicted on **concurrent communication diagrams**, which depict the concurrency viewpoint of the system (Douglass 2004). On a concurrent communication diagram, a UML 2 active object is depicted as a rectangular box with two vertical parallel lines on the left- and right-hand sides; a passive object is depicted as a regular rectangular box. The UML 1.x notation for active objects – rectangular boxes with thick black lines – is no longer used. An example is given in Figure 2.10, which also shows the notation for multiobjects (useful UML 1.x notation although no longer used in UML 2), used when more than one object is instantiated from the same class.

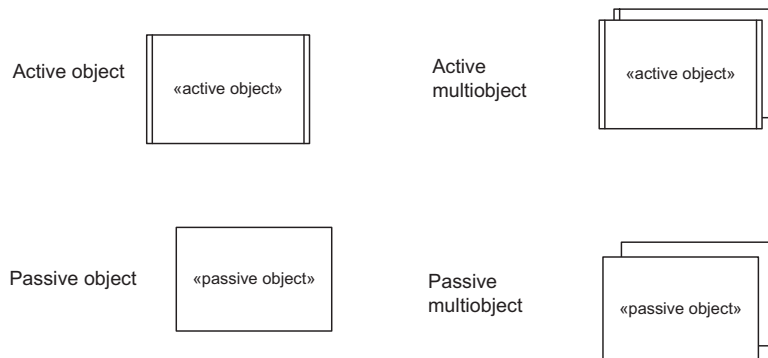


Figure 2.10. UML notation for active and passive objects

2.8.1 Message Communication on Concurrent Communication Diagrams

Message interfaces between tasks on concurrent communication diagrams are either **asynchronous** (loosely coupled) or **synchronous** (tightly coupled).

The UML notation for message communication is summarized in Figure 2.11. Figure 2.12 depicts a concurrent communication diagram, a version of the communication diagram that shows active objects (concurrent objects, processes, tasks, or threads) and the various kinds of message communication between them. For synchronous message communication, two possibilities exist: (1) synchronous message communication with reply (arrow with black arrowhead for request and dashed arrow with stick arrowhead for reply) and (2) synchronous message communication without reply (arrow with black arrowhead for request). Note that from UML 1.4 onwards, the UML notation for asynchronous communication has changed from an arrow with a half arrowhead to an arrow with a stick arrowhead. Note also that showing a simple message as an arrow with a stick arrowhead is a convention used in UML 1.3 and earlier. It is useful, however, to use simple messages during analysis modeling when no decision has yet been made about the type of message communication.

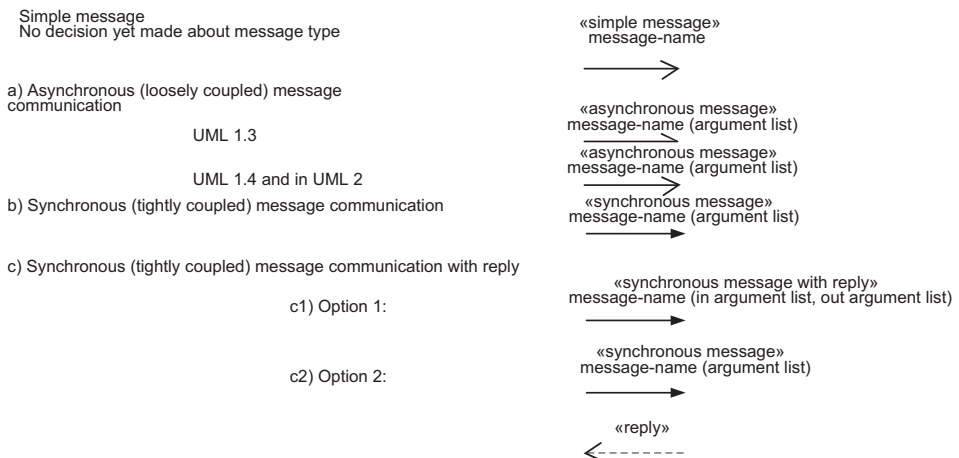


Figure 2.11. UML notation for messages

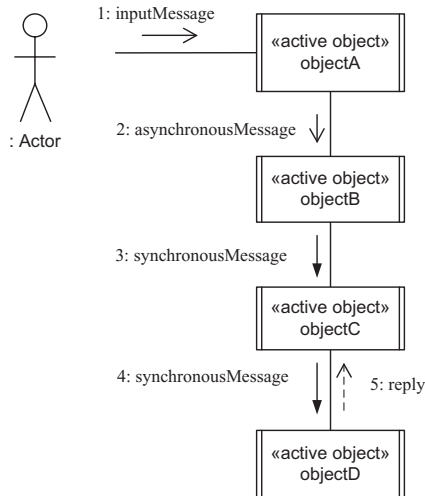


Figure 2.12. UML notation for a concurrent communication diagram

2.9 DEPLOYMENT DIAGRAMS

A **deployment diagram** shows the physical configuration of the system in terms of physical nodes and physical connections between the nodes, such as network connections. A node is shown as a cube, and the connection is shown as a line joining the nodes. A deployment diagram is essentially a class diagram that focuses on the system's nodes (Booch, Rumbaugh, and Jacobson 2005).

In this book, a node usually represents a computer node, with a constraint (see Section 2.10.3) describing how many instances of this node may exist. The physical connection has a stereotype (see Section 2.10.1) to indicate the type of connection, such as «local area network» or «wide area network». Figure 2.13 shows two examples of deployment diagrams. In the first example, nodes are connected via a wide area network (WAN); in the second, they are connected via a local area network (LAN). In the first example, the ATM Client node (which has one node for each ATM) is connected to a Bank Server that has one node. Optionally, the objects that reside at the node may be depicted in the node cube. In the second example, the network is shown as a node cube. This form of the notation is used when more than two computer nodes are connected by a network.

2.10 UML EXTENSION MECHANISMS

UML provides three mechanisms to allow the language to be extended (Booch, Rumbaugh, and Jacobson 2005; Rumbaugh, Booch, and Jacobson 2005). These are stereotypes, tagged values, and constraints.

2.10.1 Stereotypes

A **stereotype** defines a new building block that is derived from an existing UML modeling element but tailored to the modeler's problem (Booch,

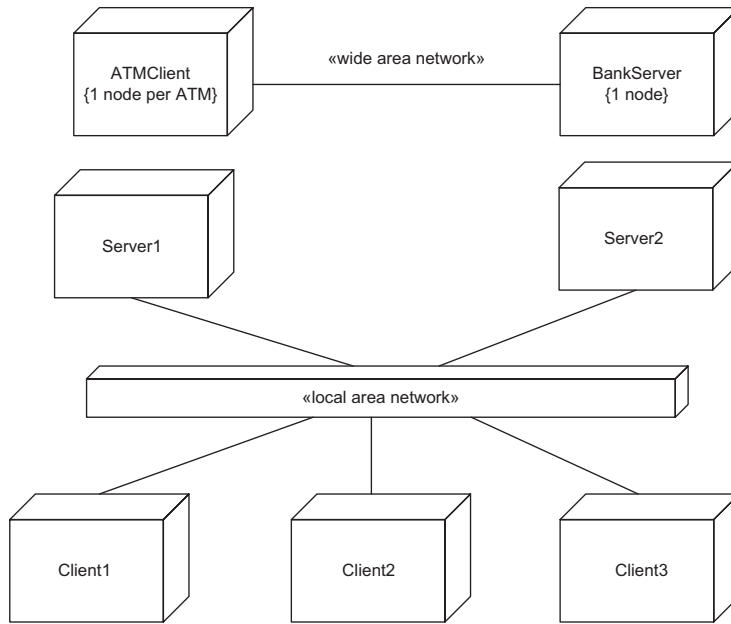


Figure 2.13. UML notation for a deployment diagram

Rumbaugh, and Jacobson 2005). This book makes extensive use of stereotypes. Several standard stereotypes are defined in UML. In addition, a modeler may define new stereotypes. This chapter includes several examples of stereotypes, both standard and COMET-specific. Stereotypes are indicated by guillemets (« »).

Figure 2.9 shows the stereotypes «system» and «subsystem» to distinguish between two different kinds of packages. Figure 2.11 uses stereotypes to distinguish among different kinds of messages.

In UML 1.3, a UML modeling element could be depicted only with one stereotype. However, UML 1.4 onward extended the stereotype concept to allow a modeling element to be depicted by more than one stereotype. Therefore, different, possibly orthogonal, characteristics of a modeling element can now be depicted with different stereotypes. The COMET method takes advantage of this additional functionality.

The UML stereotype notation allows a modeler to tailor a UML modeling element to a specific problem. In UML, stereotypes are enclosed in guillemets usually within the modeling element (e.g., class or object), as depicted in Figure 2.14a. However, UML also allows stereotypes to be depicted as symbols. One of the most common such representations was introduced by Jacobson (1992) and is used in the Unified Software Development Process (USDP) (Jacobson, Booch, and Rumbaugh 1999). Stereotypes are used to represent «entity» classes, «boundary» classes, and «control» classes. Figure 2.14b depicts the Process Plan «entity» class, the Elevator Control «control» class, and the Sensor Interface «boundary» class using the USDP's stereotype symbols.

Alternative a (standard UML notation for depicting stereotypes):



Alternative b (stereotypes used in Unified Software Development Process):



Figure 2.14. Alternative notations for UML stereotypes

2.10.2 Tagged Values

A **tagged value** extends the properties of a UML building block (Booch, Rumbaugh, and Jacobson 2005), thereby adding new information. A tagged value is enclosed in braces in the form {tag = value}. Commas separate additional tagged values. For example, a class may be depicted with the tagged values {version = 1.0, author = Gill}, as shown in Figure 2.15.

2.10.3 Constraints

A **constraint** specifies a condition that must be true. In UML, a constraint is an extension of the semantics of a UML element to allow the addition of new rules or modifications to existing rules (Booch, Rumbaugh, and Jacobson 2005). For example, for the Account class depicted in Figure 2.15, the constraint on the attribute balance is that the balance can never be negative, depicted as {balance >= 0}. Optionally, UML provides the Object Constraint Language (Warmer and Kleppe 1999) for expressing constraints.

2.11 CONVENTIONS USED IN THIS BOOK

For improved readability, the conventions used for depicting names of classes, objects, and so on in the figures are sometimes different from the conventions used for the same names in the text. In the figures, examples are shown in Times Roman

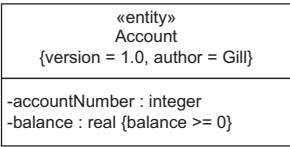


Figure 2.15. UML notation for tagged values and constraints

font. In the body of the text, however, examples are shown in a different font to distinguish them from the regular Times Roman font. Some specific additional conventions used in the book vary depending on the phase of the project. For example, the conventions for capitalization are different in the analysis model (which is less formal) than in the design model (which is more formal).

2.11.1 Requirements Modeling

In both figures and text, use cases are shown with an uppercase initial letter and spaces in multiword names – for example, Withdraw Funds.

2.11.2 Analysis Modeling

The naming conventions for the analysis model are as follows.

Classes

Classes are shown with an uppercase initial letter. In the figures, there are no spaces in multiword names – for example, CheckingAccount. In the text, however, spacing is introduced to improve the readability – for example, Checking Account.

Attributes are shown with a lowercase initial letter – for example, balance. For multiword attributes, there are no spaces between the words in figures, but spaces are introduced in the text. The first word of the multiword name has an initial lowercase letter; subsequent words have an initial uppercase letter – for example, account-Number in figures, and account Number in text.

The type of the attribute has an initial uppercase letter – for example, Boolean, Integer, or Real.

Objects

Objects may be depicted in various ways, as described next:

- **An individual named object.** In this case, the first letter of the first word is lowercase, and subsequent words have an uppercase first letter. In figures, the objects appear as, for example, aCheckingAccount and anotherCheckingAccount. In the text, these objects appear as a Checking Account and another Checking Account.
- **An individual unnamed object.** Some objects are shown in the figures as class instances without a given object name – for example, : CheckingAccount. In the text, this object is referred to as Checking Account. For improved readability, the colon is removed, and a space is introduced between the individual words of a multiword name.

This means that, depending on how the object is depicted in a figure, it will appear in the text sometimes with a first word in which the initial letter is uppercase and sometimes with a first word in which the initial letter is lowercase.

Messages

In the analysis model, messages are always depicted as simple messages (see [Figure 2.11](#) and Section 2.8.1) because no decision has yet been made about the message type. Messages are depicted with an uppercase initial letter. Multiword messages are shown with spaces in both figures and text – for example, Simple Message Name.

Statecharts

In both figures and text, states, events, conditions, actions, and activities are all shown with initial letter uppercase and spaces in multiword names – for example, the state Waiting for PIN, the event Cash Dispensed, and the action Dispense Cash.

2.11.3 Design Modeling

The naming conventions for the design model are as follows.

Active and Passive Classes

The naming conventions for active classes (concurrent classes) and passive classes are the same as for classes in the analysis model (see Section 2.11.2).

Active and Passive Objects

The naming conventions for active objects (concurrent objects) and passive objects are the same as for objects in the analysis model (see Section 2.11.2).

Messages

In the design model, the first letter of the first word of the message is lowercase, and subsequent words have an uppercase first letter. In both the figures and text, there is no space between words, as in `alarmMessage`.

Message parameters are shown with a lowercase initial letter – for example, `speed`. For multiword attributes, there are no spaces between the words in both the figures and text. The first word of the multiword name has a lowercase initial letter, and subsequent words have an uppercase initial letter – for example, `cumulativeDistance` – in both figures and text.

Operations

The naming conventions for operations (also known as methods) follow the conventions for messages in both figures and text. Thus, the first letter of the first word of both the operation and the parameter is lowercase, and subsequent words have an uppercase first letter. There is no space between words – for example, `validatePassword` (`userPassword`).

2.12 SUMMARY

This chapter briefly described the main features of the UML notation and the main characteristics of the UML diagrams used in this book.

For further reading on UML 2 notation, Fowler (2004) and Ambler (2005) provide introductory material. More detailed information can be found in Booch,

Rumbaugh, and Jacobson (2005) and Eriksson et al. (2004). A comprehensive and detailed reference to UML is Rumbaugh, Booch, and Jacobson (2005).

EXERCISES

Multiple-choice questions: For each question, choose one of the answers.

1. How is an actor depicted on a use case diagram?
 - (a) An oval
 - (b) A stick figure
 - (c) A box
 - (d) A dashed line
2. How is a use case depicted on a use case diagram?
 - (a) An oval
 - (b) A stick figure
 - (c) A box
 - (d) A dashed line
3. How is a class depicted on a class diagram?
 - (a) A box with one compartment
 - (b) A box with one or two compartments
 - (c) A box with one, two, or three compartments
 - (d) An oval
4. How is an association depicted on a class diagram?
 - (a) A solid line joining two class boxes
 - (b) A dashed line joining two class boxes
 - (c) A diamond touching the upper class box
 - (d) An arrowhead touching the upper class box
5. How is public visibility depicted for a class element on a class diagram?
 - (a) + sign
 - (b) – sign
 - (c) # sign
 - (d) *sign
6. What are the two kinds of UML interaction diagrams?
 - (a) Class diagram and sequence diagram
 - (b) Sequence diagram and communication diagram
 - (c) Class diagram and communication diagram
 - (d) Statechart and communication diagram
7. What does an interaction diagram depict?
 - (a) Objects and links
 - (b) Classes and relationships
 - (c) Objects and messages
 - (d) States and events
8. What does a statechart diagram depict?
 - (a) Objects and links
 - (b) Classes and relationships
 - (c) Objects and messages
 - (d) States and events
9. What is a UML package?
 - (a) A box
 - (b) A grouping of classes
 - (c) A grouping of use cases
 - (d) A grouping of model elements
10. What does a deployment diagram depict?
 - (a) The physical configuration of the system in terms of physical classes and physical connections between the classes
 - (b) The physical configuration of the system in terms of physical objects and physical connections between the objects
 - (c) The physical configuration of the system in terms of physical nodes and physical connections between the nodes
 - (d) The physical configuration of the system in terms of physical computers and physical networks between the computers